LMPool

Locality-Aware Routing and NVLink KV-Cache Transfer for Multi-GPU LLM Serving

Jialiang Li

The Hong Kong University of Science and Technology

July 28th 2026

https://github.com/0x0addc001/LMPool

The Placement Mismatch

What prefix caching assumes

Reuse is cheap when a request executes where its KV prefix already resides.

What data parallelism does

Each GPU owns an independent allocator and physical KV cache, while ingress assigns requests globally.

- Round robin may send a matching request to the wrong GPU.
- Strict owner routing can overload one replica.
- Unconditional KV movement can cost more than recomputation.

Design question

When should the system route to existing KV, and when should KV move to demand?

Two Principles

Routing Principle: Cache Locality

Send a request to a GPU that already owns the longest useful contiguous prefix when the saved prefill work exceeds queueing and capacity pressure.

Transfer Principle: Cache Fluidity

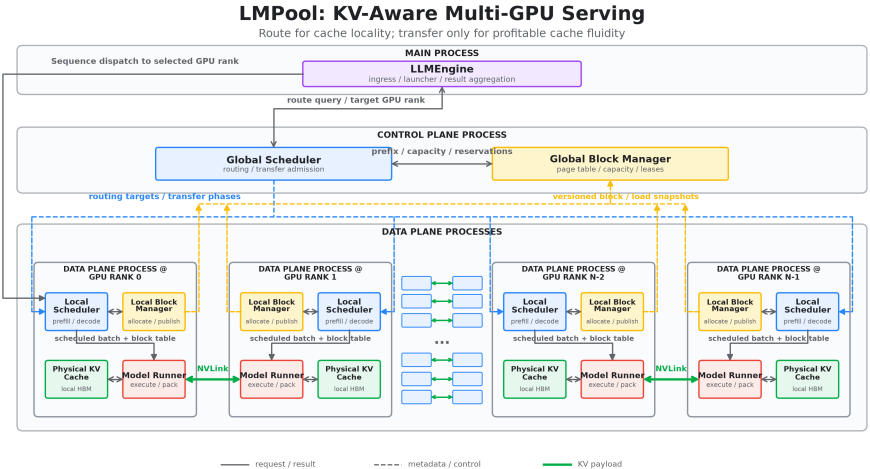
Copy or move complete KV block chains over a direct NVLink pair when cache placement no longer matches future demand and expected reuse repays the transaction.

Route before transfer

Transfer only when profitable

Avoid recomputation without turning locality into a load hotspot

System Architecture



A Request Through LMPool

1. **Ingress:** LLMEngine tokenizes the prompt, builds a Sequence, and computes chained hashes for complete prompt blocks.
2. **Global route:** Global Scheduler reads versioned prefix, load, and capacity metadata and reserves the selected rank.
3. **Local admission:** The target Local Scheduler reuses ready local blocks and asks Local Block Manager to allocate missing blocks.
4. **Optional foreground transfer:** A real block shortage may trigger a cost-gated pair-local copy or move plan.
5. **Execution:** Model Runner performs prefill, writes missing KV, and then iterates decode.
6. **Publication:** The worker reports tokens and a new versioned block/load snapshot, only ready blocks become globally visible.
7. **Completion:** References are released. Complete prefixes remain cached at ref-count zero until reuse or dependency-safe reclamation.

KV-Aware Routing

For candidate GPU g:

$$
\begin{aligned}
n_g &= \max(0, R - h_g), \\
M_g &= \min(N, n_g S), \\
A_g &= \max(0, n_g - F_g), \\
C_{\text{route}}(g) &= Q_g + w_p M_g + w_e A_g S.
\end{aligned}
$$

Decision

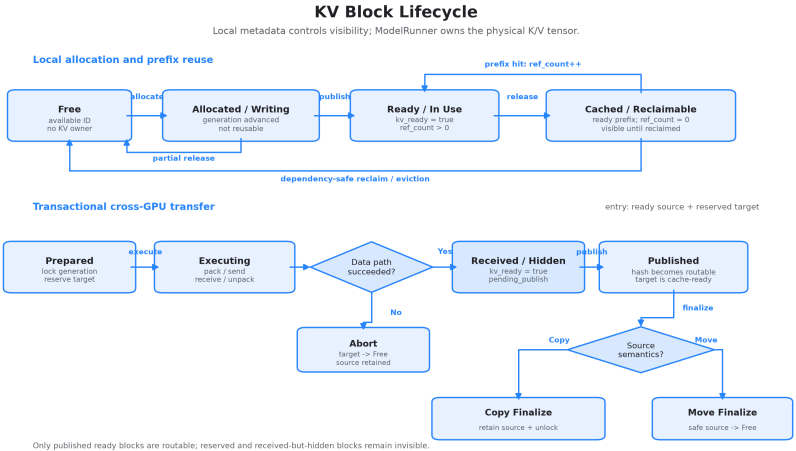
Select the feasible rank with minimum projected token-equivalent work.

- R: required prompt blocks, N: prompt tokens
- S: tokens per block, h_g: contiguous ready blocks
- n_g: missing blocks, M_g: missing prefill tokens
- F_g: currently free blocks
- A_g: missing blocks beyond F_g, hence reclaim pressure
- Q_g: queued, pending, and running token-equivalent work
- w_p, w_e: prefill and reclaim cost weights

Bounded spill

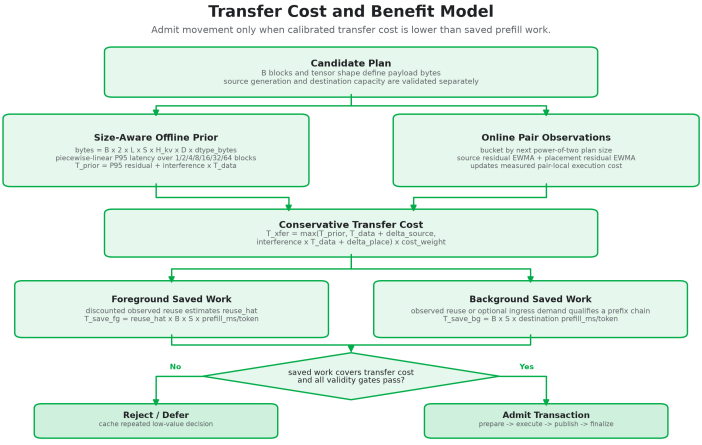
Bypass an overloaded owner only when additional recomputation remains bounded.

Transactional KV Transfer



Prepare → Execute → Publish → Finalize, abort preserves the source and releases the destination reservation.

Cost-Gated Transfer Admission



Static cost uses the pair-specific P95 profile plus a calibrated residual. Fixed latency is only a zero-default fallback, completed plans update EWMA residuals.

Evaluation Design

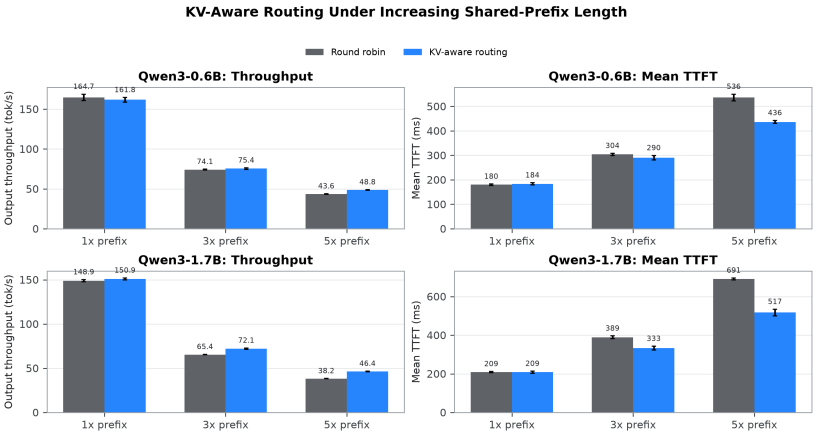
- 6 RTX 3090 GPUs, 3 direct NV4 pairs
- Qwen3-0.6B and Qwen3-1.7B, BF16
- Same per-worker KV budget
- 5 trials, fixed seed, EOS disabled
- Outputs: routing 64, memory skew 16, load skew 8 tokens
- Controlled internal ablations

Workload	Purpose
Routing	1x/3x/5x shared-prefix locality
Load skew	Background copy plus placement leases
Memory skew	Move-style capacity release

Scope

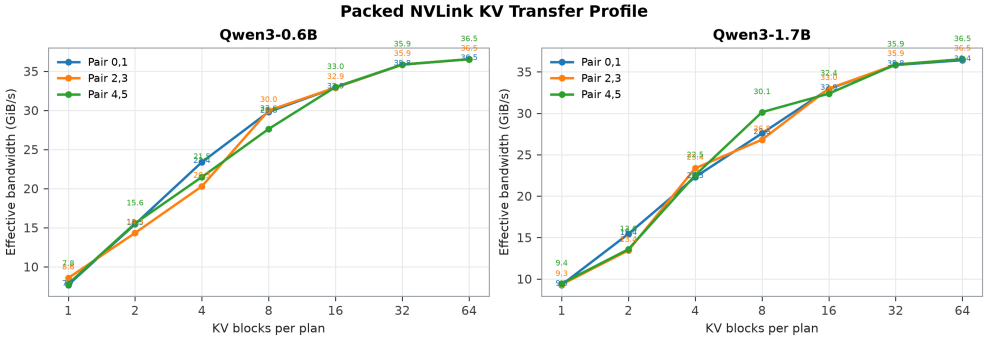
Synthetic mechanism study, not a production-trace or production-engine comparison.

Routing: Longer Prefixes Create Stable Gains



At 5x, routing improves throughput by **11.8% / 21.3%** and reduces TTFT by **18.6% / 25.2%** for 0.6B / 1.7B.

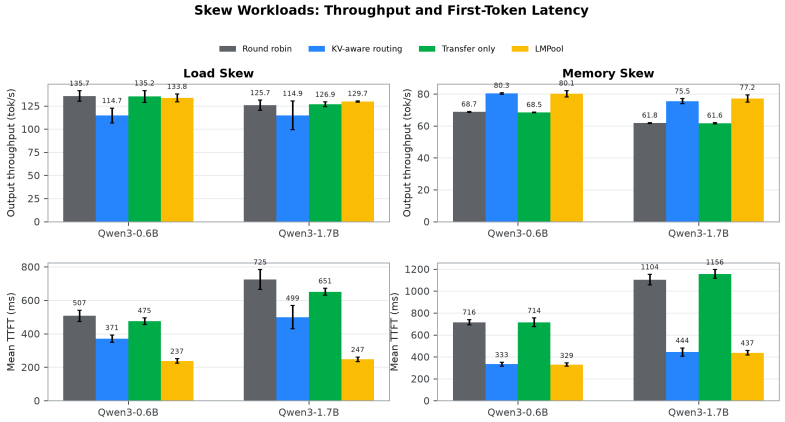
NVLink Data-Path Calibration



Interpretation

Every destination passed byte validation. The 1-64 block curves calibrate the pair-specific cost model, they do not alone prove serving benefit.

Transfer: Mechanism Evidence and Boundary



Load skew: three background placements copy 87 blocks. Placement leases serve 95 later requests. TTFT falls sharply, but a higher TPOT can offset mean E2E.

Memory skew: 0.6B verifies 35 released source blocks. It remains close to routing-only, the 1.7B run does not satisfy the full offload verification predicate.

Limitations and Future Work

- Synthetic workloads and deterministic replay order, no production trace evaluation
- Internal ablations rather than production-engine comparisons
- Six RTX 3090 GPUs, two sub-2B models with identical KV geometry
- No NVSwitch-fabric evaluation, pair-local profiles do not capture all-to-all target selection or shared-fabric contention
- Fixed 256-token block size, no block-size sensitivity study
- Pair-specific P95 profiles and online residuals, no held-out cost-prediction error or policy ablation
- Single control process and launcher, no consensus or launcher failover

Next evaluation

Production traces and baselines, 1/2/4/6-GPU and block-size scaling, cost-model prediction and admission ablations, concurrent transfer profiles and end-to-end scaling on an eight-GPU NVSwitch domain.

Takeaways

1. **Routing solves cache locality:** reuse KV where it already resides without creating a load hotspot.
2. **NVLink transfer provides cache fluidity:** move complete block chains only when reuse repays the full transaction.
3. **Transfer mechanisms are verified:** background copy, placement leases, and safe source-block release execute under their respective stressors.
4. **The boundary is part of the result:** a completed transfer does not automatically improve mean E2E or throughput.

Route when possible. Transfer when profitable.

Source: `https://github.com/0x0addc001/LMPool`

Thank You for Listening!

Questions?

Acknowledgments

Thank Prof. Kai Chen and Yijun Sun for their guidance and feedback.

Jialiang Li The Hong Kong University of Science and Technology

`https://github.com/0x0addc001/LMPool`